



Sensors Data Provider

Sensor data transfer between iCUE and HTML widgets: sensor types, units, values, device names, and more.

Overview

- **Module name:** widgetbuilder.sensorsdataprovder
- **Plugin name:** Sensors
- **Version:** 1.0

Manifest entry:

```
"required_plugins": [  
  "widgetbuilder.sensorsdataprovder:Sensors:1.0"  
]
```

All data retrieval methods are asynchronous using `requestId` for correlation.

Async Request Pattern

1. Call a method with unique `requestId`
2. Listen for `asyncResponse(requestId, value)` signal
3. Match response `requestId` to request



```
let nextRequestId = 0;
const requestId = nextRequestId++;
window.plugins.Sensorsdataprotider.asyncResponse.connect((id, value) => {
    if (id === requestId) {
        console.log("Sensor value:", value);
    }
});
window.plugins.Sensorsdataprotider.getSensorValue(requestId, sensorId);
```

Methods

getSensorValue(requestId, sensorId)

Gets current sensor value.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: string

getSensorUnits(requestId, sensorId)

Gets units of measurement.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: string (e.g., "C", "%", "RPM")

getSensorName(requestId, sensorId)

Gets sensor display name.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: string

getSensorDeviceName(requestId, sensorId)

Gets device name containing the sensor.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: string

getSensorType(requestId, sensorId)

Gets sensor type.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: string - See [sensor types](#)

getSensorKind(requestId, sensorId)

Gets sensor kind/category.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: string - See [sensor kind](#)

getAllSensorIds(requestId)

Gets all available sensor identifiers.

Parameter	Type	Description
requestId	int	Request ID

Response: string[]

sensorIsConnected(requestId, sensorId)

Checks sensor availability.

Parameter	Type	Description
requestId	int	Request ID
sensorId	string	Sensor identifier

Response: bool

`getDefaultSensorId(requestId, sensorType, preferredSensorKind)`

Gets default sensor ID for a given type with an optional preferred kind hint.

The lookup follows this priority order:

1. First sensor matching both `sensorType` and `preferredSensorKind`
2. First sensor matching `sensorType` only (if no kind match found)
3. First available sensor of any type (if no type match found)

If `preferredSensorKind` is an empty string, the kind is not used as a filter and the first sensor matching `sensorType` is returned.

Parameter	Type	Description
<code>requestId</code>	<code>int</code>	Request ID
<code>sensorType</code>	<code>string</code>	Sensor type (see sensor types)
<code>preferredSensorKind</code>	<code>string</code>	Preferred sensor kind (see sensor kind). Pass <code>""</code> to match any kind.

Response: `string` - Sensor ID, or empty string if no sensors are available

`getDefaultSensorIdBlock(sensorType, preferredSensorKind)`

Synchronously gets default sensor ID for a given type with an optional preferred kind hint.
Blocking call.

Applies the same lookup priority as `getDefaultSensorId`.

Parameter	Type	Description
<code>sensorType</code>	<code>string</code>	Sensor type (see sensor types)

Parameter	Type	Description
<code>preferredSensorKind</code>	<code>string</code>	Preferred sensor kind (see sensor kind). Pass <code>""</code> to match any kind.

Returns: `string` - Sensor ID, or empty string if no sensors are available

Signals

`asyncResponse(requestId, value)`

Emitted when async method completes.

Parameter	Type	Description
<code>requestId</code>	<code>int</code>	Original request ID
<code>value</code>	<code>var</code>	Response value

`sensorAdded(sensorId)`

Emitted when new sensor becomes available.

Parameter	Type	Description
<code>sensorId</code>	<code>string</code>	Added sensor ID

`sensorRemoved(sensorId)`

Emitted when sensor is no longer available.

Parameter	Type	Description
sensorId	string	Removed sensor ID

sensorDataChanged(sensorId)

Emitted when sensor data changes.

Parameter	Type	Description
sensorId	string	Changed sensor ID

sensorValueChanged(sensorId, value)

Emitted when sensor value changes.

Parameter	Type	Description
sensorId	string	Sensor ID
value	string	New sensor value

sensorUnitsChanged(sensorId, units)

Emitted when sensor units change (e.g., Celsius to Fahrenheit).

Parameter	Type	Description
sensorId	string	Sensor ID
units	string	New units

SimpleSensorApiWrapper

Promise-based wrapper for the Sensors plugin. Converts the callback-based Qt async API into Promises.

Important: Use Local Wrapper Files

Copy `common/plugins/` from the documentation bundle into your widget folder and include wrappers via local `<script src>` paths.

Do not reference iCUE installation paths (for example `../common/plugins/...`) in third-party widgets.

Initialization

```
const api = new SimpleSensorApiWrapper(sensorsplugin);
```

Parameter	Type	Default	Description
<code>plugin</code>	<code>object</code>	-	Plugin instance (<code>window.plugins.Sensorsdataproducer</code>)
<code>timeoutMs</code>	<code>number</code>	<code>5000</code>	Request timeout (ms)

Methods

All methods return a `Promise`.

Method	Returns	Description
<code>getSensorValue(sensorId)</code>	<code>Promise<string></code>	Current sensor reading
<code>getSensorUnits(sensorId)</code>	<code>Promise<string></code>	Units string (e.g. <code>"°C"</code> , <code>"RPM"</code>)

Method	Returns	Description
<code>getSensorName(sensorId)</code>	<code>Promise<string></code>	Human-readable sensor name
<code>getSensorDeviceName(sensorId)</code>	<code>Promise<string></code>	Device the sensor belongs to
<code>getSensorType(sensorId)</code>	<code>Promise<string></code>	Sensor type identifier
<code>getSensorKind(sensorId)</code>	<code>Promise<string></code>	Sensor kind
<code>getAllSensorIds()</code>	<code>Promise<string[]></code>	All available sensor IDs
<code>sensorIsConnected(sensorId)</code>	<code>Promise<boolean></code>	Whether sensor is active

Required local files

```
<script src="common/plugins/IcueWidgetApiWrapper.js"></script>
<script src="common/plugins/SimpleSensorApiWrapper.js"></script>
```

Example

```
`manifest.json`

```json
"required_plugins": [
 "widgetbuilder.sensorsdatapvider:Sensors:1.0"
]
```

index.html

```
// After including IcueWidgetApiWrapper and SimpleSensorApiWrapper in <head>:
const sensorApi = new SimpleSensorApiWrapper(window.plugins.Sensorsdatapvider);

async function displayAllSensors() {
 const sensorIds = await sensorApi.getAllSensorIds();
 for (const sensorId of sensorIds) {
 const [name, value, units] = await Promise.all([
 sensorApi.getSensorName(sensorId),
 sensorApi.getSensorValue(sensorId),
 sensorApi.getSensorUnits(sensorId),
]);
 }
}
```

```
 });
 console.log(`${name}: ${value} ${units}`);
 }
}
displayAllSensors();
```

# Appendixes

## Sensor Types

Value	Description
temperature	Temperature
pump	Pump speed
fan	Fan speed
voltage	Voltage
load	Load/usage
led	LED (deprecated)
cas-latency	CAS latency (RAM)
command-rate	Command rate (RAM)
cycle-time	Cycle time (RAM)
dram-frequency	DRAM frequency
ras-precharge	RAS precharge (RAM)
ras-to-cas-delay	RAS to CAS delay (RAM)

Value	Description
current	Current
power	Power
battery-charge	Battery charge level
battery-status	Battery status
efficiency	Efficiency
fps	Frames per second
pin-protect	Pin protection

## Sensor Kind

Value	Description
default	Default
core	Processor core
package	CPU package
power-in	Input power
power-out	Output power
power-3-3	3.3V power
power-5	5V power
power-12	12V power

Value	Description
total-power-draw	Total power consumption
voltage-bat	Battery voltage
voltage-core	Core voltage
voltage-in	Input voltage
voltage-3-3	3.3V voltage
voltage-5	5V voltage
voltage-12	12V voltage
voltage-vdd	VDD voltage (RAM)
voltage-vddq	VDDQ voltage (RAM)
voltage-vpp	VPP voltage (RAM)
current-3-3	3.3V current
current-5	5V current
current-item-12v	12V item current
current-item-12v-2x6	12V 2x6 current
current-12v	12V current
current-item-atx	ATX current
current-item-sata	SATA current
cpu-temp	CPU temperature

Value	Description
gpu-temp	GPU temperature
cpu-pump	CPU pump
gpu-pump	GPU pump
gpu-load	GPU load
pin-protect-status	Pin protection status
pin-protect-current	Pin protection current
pin-protect	Pin protection
memory-load	Memory load
frame-buffer-load	Frame buffer load
video-engine-load	Video engine load
bus-interface-load	Bus interface load
invalid	Invalid kind